

COMP2054-ADE

ADE Lec06

Linked Lists

Lecturer: Andrew Parkes

<http://www.cs.nott.ac.uk/~pszajp/>

Relevance

- It is generally assumed that you will have done singly and doubly linked lists
 - Please revise them, if not then these slides just cover the basic ideas
- Our goal in this module is to be more careful about their efficiency, that is, to observe the complexity, $O(1)$ versus $O(n)$, of the various standard operations
 - Where ' n ' is the length of the list
- Also, lists are relevant to “simple sorting” algorithms, and to “stacks” and “queues”

Standard Operations

For (almost) all data structures need to implement:

- “Find”: Does the list contain some element?
- “Insert”: Insert a new element
- “Delete”:
 - Remove some element with a given position, or
 - Remove some element with a given value

Want them to be efficient – as measured by Big-Oh

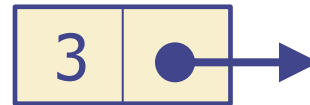
Singly Linked List: Node

Each node in a singly linked list has

- Pointer/Reference to the next node
- Stores some data, "D":
 - Directly as some primitive data type, or
 - As a Pointer (reference) to some data object

E.g.

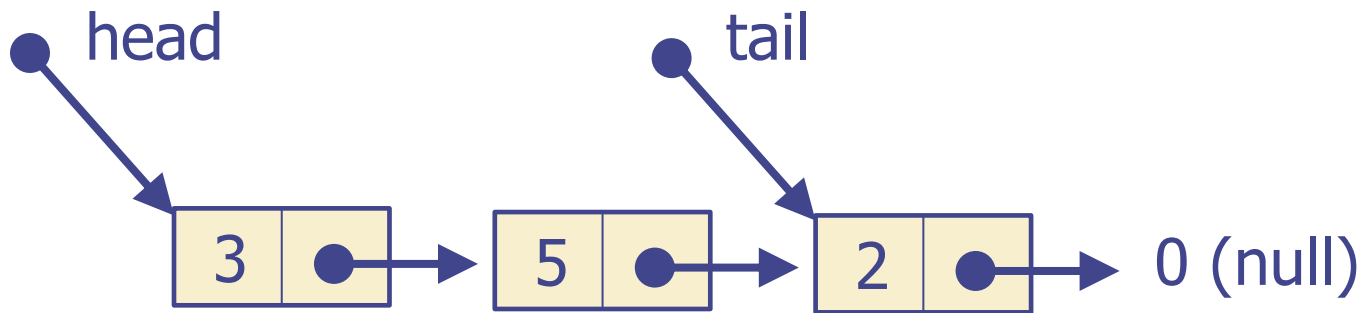
```
Node {  
    Node*    next;  
    int      k; // or can be D* d  
}
```



Singly Linked List: List

List contains a pointer, 'head' to the start of a list. Often it will also have a pointer to the 'tail'.

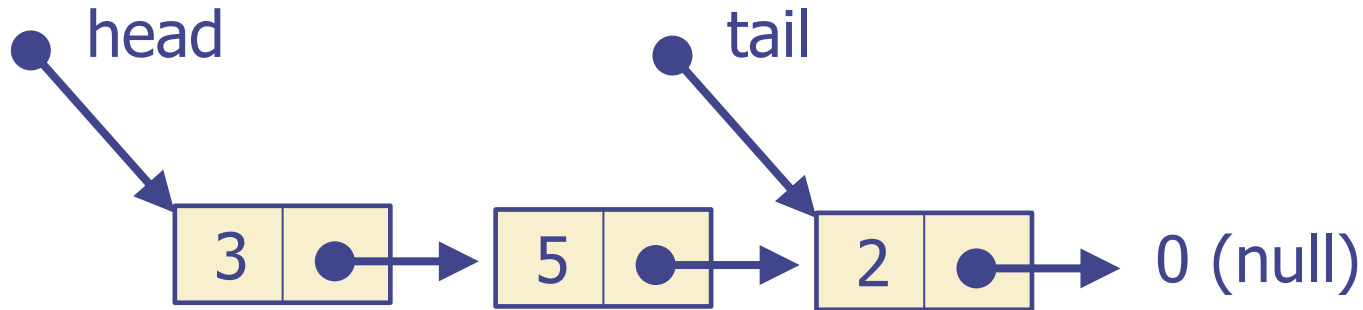
```
List {  
    Node*    head;  
    Node*    tail;  
    int      n; // number of elements  
}
```



Usage?

- In a simple linked list, all the data is accessible from the head by just “walking along the list”
 - Hence, it is clear (hopefully) that all “standard” operations (insert/delete etc) are implementable
- The key question is what operations are implementable **efficiently**!?
 - Need $O(n)$ of operations in terms of the length n of the list
 - Note: Typically the overall length of a list (or other data structure) is stored as an auxiliary element to allow fast access as it is generally used a lot.
 - Note: There is no direct access to the middle of a list - unlike an array which has “random (arbitrary) access” in $O(1)$.

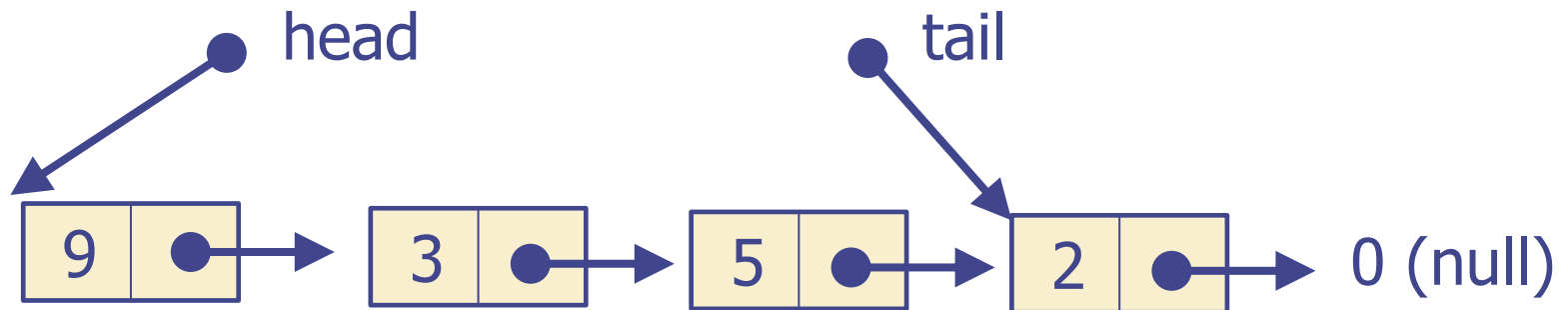
Insert at head



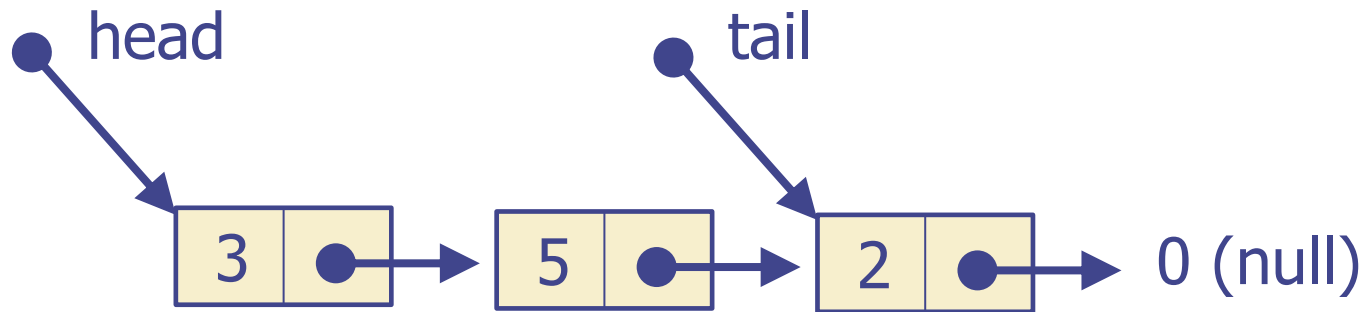
Insert '9':

1. Create a new node N containing '1'
2. Point N.head at old head
3. Point head at N

Clearly $O(1)$ as does not even access the n elements



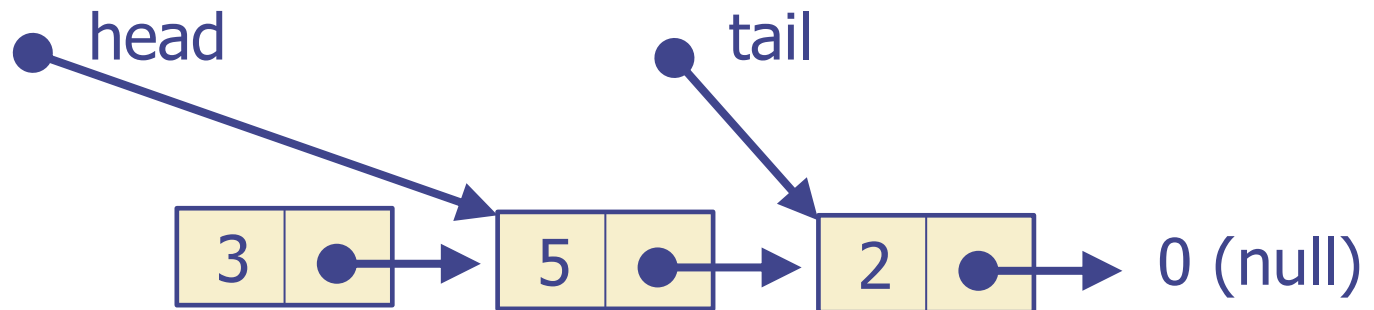
Remove at head



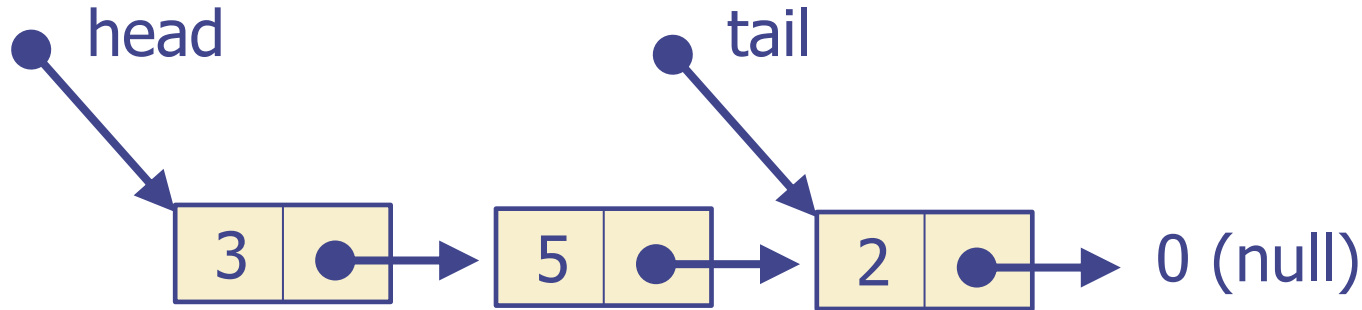
Remove head:

1. From the head node get the head.next
2. Point 'head' to the old head.next
3. Delete old head node, or let garbage collector

Clearly $O(1)$ as does not even access the n elements



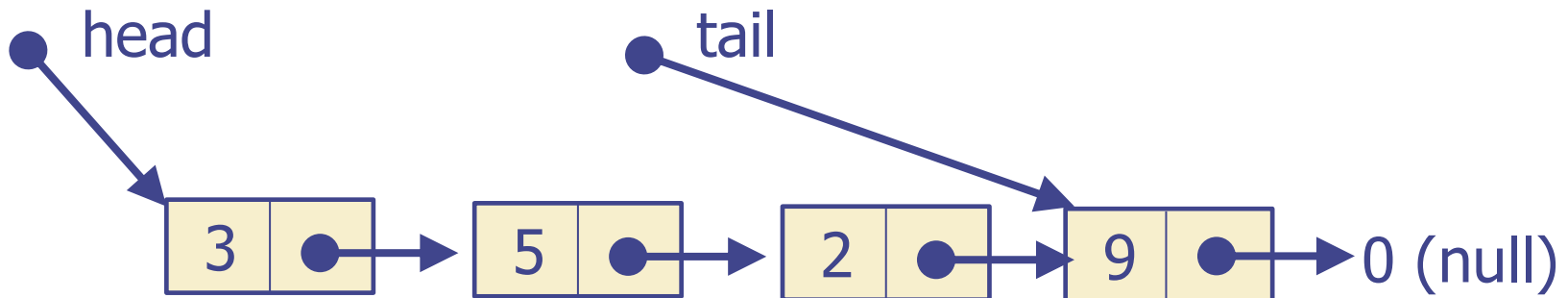
Insert at tail



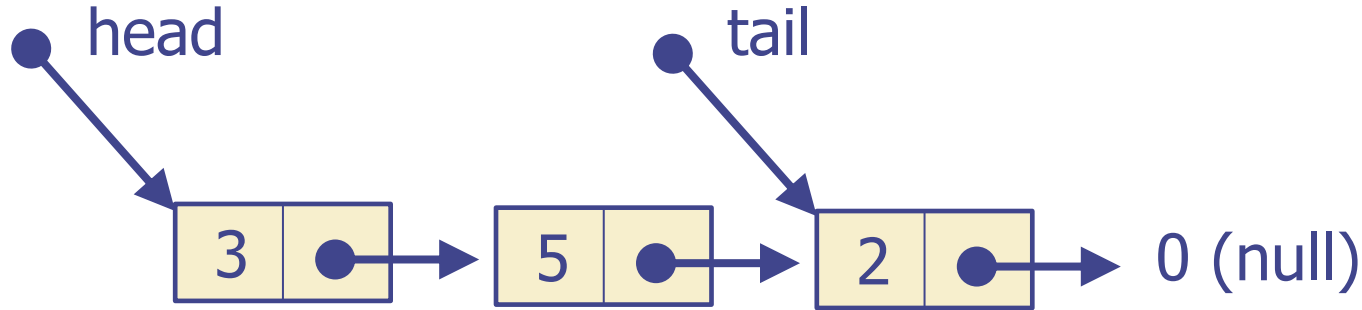
Insert '9':

1. Create a new node N containing '9'
2. Point tail.next at N
3. Point tail at N

Clearly $O(1)$ as does not even access the n elements.
Would be $O(n)$ if do not have the tail pointer.



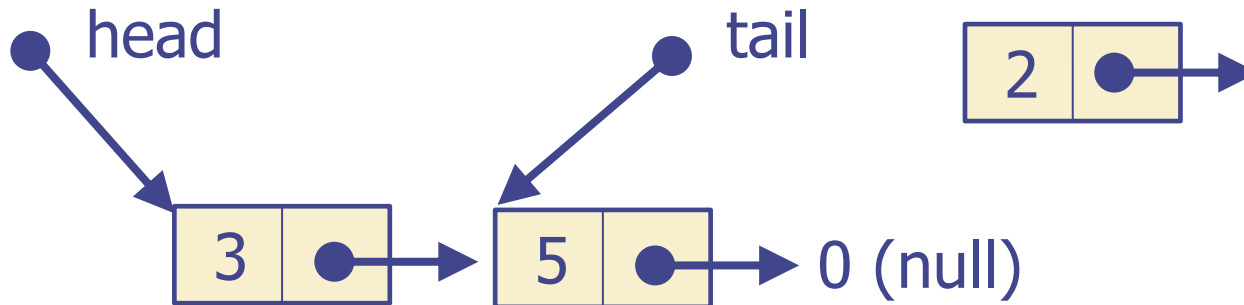
Remove at tail



We need to point the tail at the node '5' before the '2'

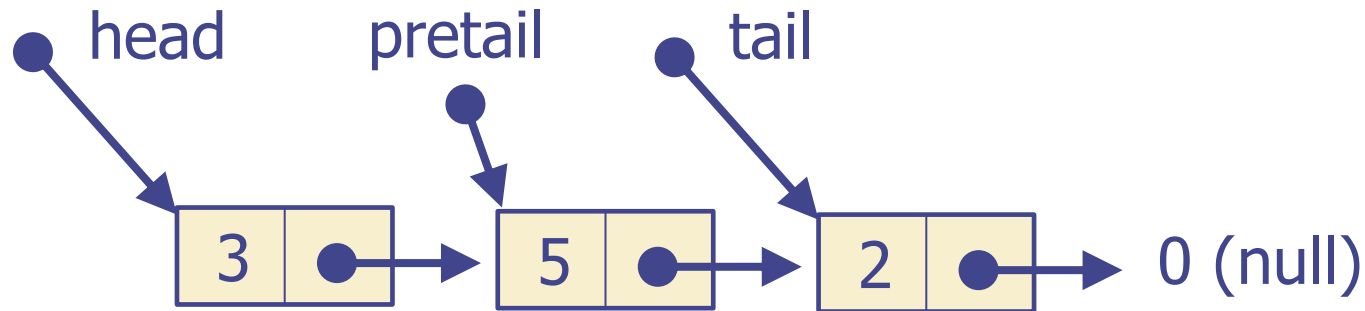
But there is no way to access that node without starting at the head, and walking down the list

Hence, it is $O(n)$. **Remove at the tail is not efficient**



Remove at tail: improve?

Could we help by having another pointer 'pre-tail' to the "pre-tail" node?



The catch is that to maintain the structure we would need to update 'pre-tail' and so get the same problem of having to walk from the head.

Obvious fix: Doubly linked list – each node also has a 'prev' so can walk to the previous node.

Advanced option (not needed): "Skip lists"

Doubly Linked List: Node

Each node in a doubly linked list has

- Pointer/Reference to the next node
- Pointer/Reference to the previous node
- Stores some data, "D"

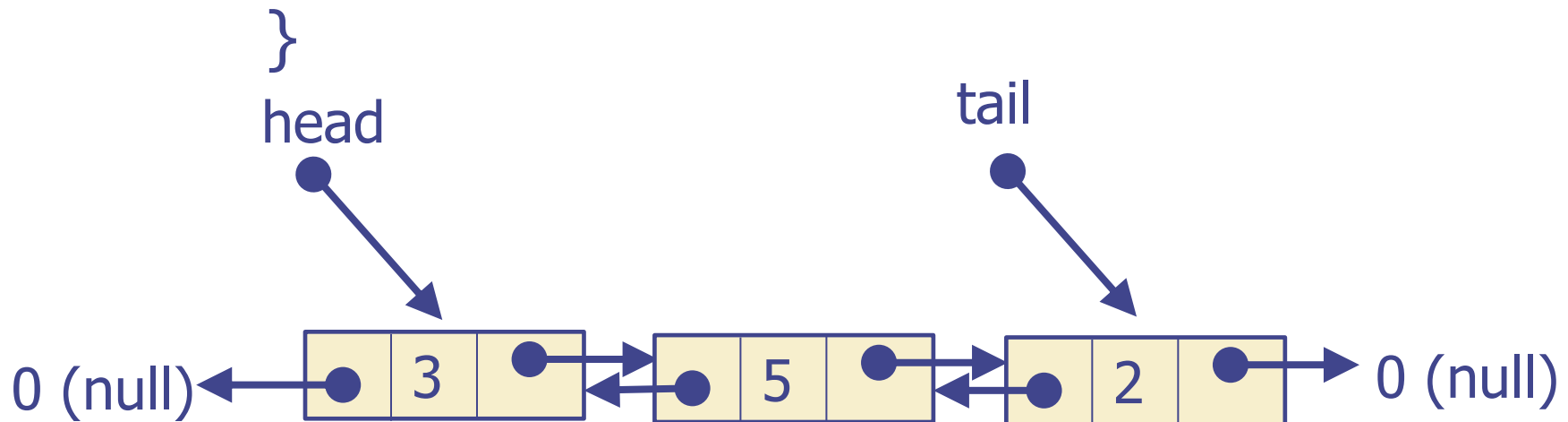
```
Node {  
    Node*    next;  
    Node*    prev;  
    int      k; // or can be D* d  
}
```



Doubly Linked List: List

IntList contains a pointer, 'head' to the start of a list. Often it will also have a pointer to the 'tail'.

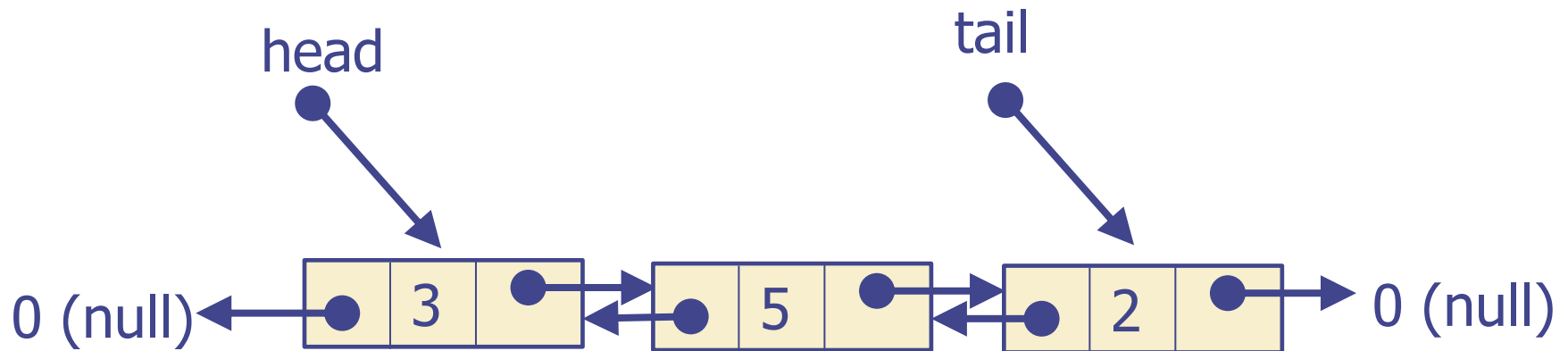
```
intList {  
    intNode*    head;  
    intNode*    tail;  
    int         n; // size  
}
```



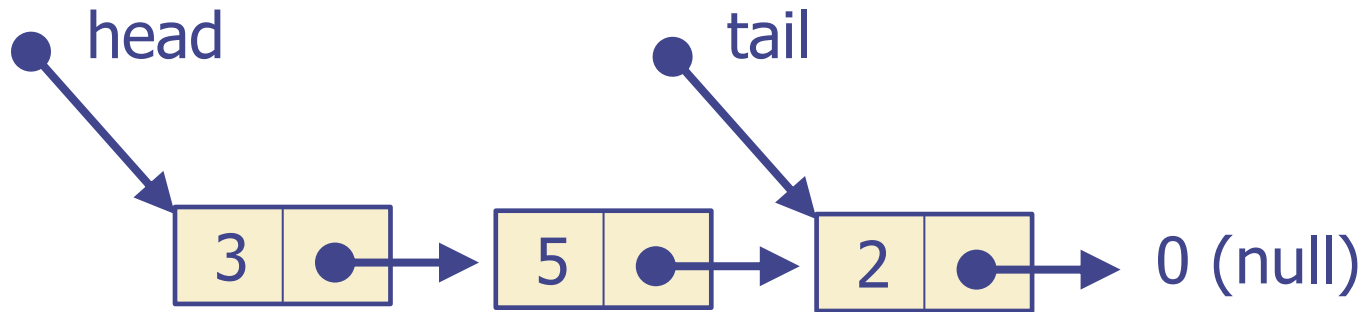
Doubly Linked List: Big-Oh?

It should be clear that all insert/delete at the head/tail are now all $O(1)$.

Exercise (offline): Check this – implement it if in the slightest doubt



Operations during processing or scanning of a list



We might require operations that can also insert and delete during the processing of a list.

E.g. “find and delete” operations e.g. `findDelete(5)`

This will need to scan and so are already $O(n)$.

However, as we scan we can keep track of which node we just came from and so will be able to do the removal of the node, or insertion of a new node $O(1)$ time. Even with just a singly-linked list.

Exercise (offline): Check this – implement it if in the slightest doubt, or ask in labs/tutorials

Singly- vs. Doubly-linked

Comparison of Singly and Doubly linked.

Singly-linked:

- Removal at the tail is much more expensive
- Only stores one pointer per node

Doubly-linked:

- Removal at tail is $O(1)$
- Stores two pointers per node.
 - More storage needed – increased cache usage
 - More maintenance

So have to select depending on the context and likely usage.
For both “processing while scanning” is likely to stay as $O(n)$.

Minimum Expectations

- It is generally assumed that you will be familiar with singly and doubly linked lists
- You should be aware of the complexity, $O(1)$ versus $O(n)$, of the various standard operations, and how to implement them.
- Also, how one would do insert/delete/etc whilst in the process of scanning